

ЗВІТ
до лабораторної роботи №1
«Основи функціонального програмування у Python»

Варіант 1. Обробка замовлень інтернет-магазину

1. Мета роботи

Ознайомитися з основними принципами функціонального програмування мовою Python. Навчитися створювати чисті функції, забезпечувати референтну прозорість, уникати мутації вхідних структур даних та ізолювати операції введення-виведення від обчислювального ядра програми.

Набути практичних навичок використання `typing.Callable` для передачі функцій як аргументів і створення функцій вищого порядку.

2. Постановка завдання

Для варіанта №1 необхідно розробити програму обробки замовлень інтернет-магазину.

Кожне замовлення містить:

- ідентифікатор замовлення;
- список товарів;
- ціну кожного товару;
- кількість одиниць товару;
- ознаку оплати.

Необхідно:

1. відібрати лише оплачені замовлення;
2. обчислити початкову суму кожного замовлення;
3. відфільтрувати замовлення, які не досягають мінімальної суми;
4. застосувати знижку;
5. застосувати податок;
6. сформувати новий об'єкт замовлення з полем `total`;
7. визначити загальну кількість прийнятих замовлень;
8. визначити загальну виручку.

Політики фільтрації, знижки та оподаткування необхідно передавати через `Callable`. Вхідні дані змінювати заборонено.

3. Структура проєкту

```
lab01/
├── core.py
├── app.py
├── tests/
│   └── test_core.py
├── requirements.txt
└── README.md
```

Файл `core.py` містить тільки чисте функціональне ядро. `app.py` відповідає за I/O, зокрема виведення результатів. Такий поділ відповідає вимозі методички відокремити чисті обчислення від побічних ефектів.

4. Код файлу `core.py`

У цьому коді `order_subtotal()`, `with_total()`, `is_paid()` та функції обробки не виконують `print()`, не читають файлів і не змінюють вхідні словники.

5. Код файлу `app.py`

Усі операції введення-виведення винесено в окремий модуль.

6. Приклад виконання програми

Для замовлення №1:

$1200 \times 1 + 600 \times 2 = 2400$ грн

Після знижки 10 %:

$2400 \times 0.90 = 2160$ грн

Після податку 20 %:

$2160 \times 1.20 = 2592$ грн

Для замовлення №4:

$2500 \times 1 + 1800 \times 2 = 6100$ грн

$6100 \times 0.90 = 5490$ грн

$5490 \times 1.20 = 6588$ грн

Замовлення №2 не обробляється, оскільки воно не оплачене. Замовлення №3 відхиляється, оскільки його сума 150 грн менша за встановлений мінімум 1000 грн.

Тому результат:

РЕЗУЛЬТАТ ОБРОБКИ

Замовлення №1: 2592.00 грн

Замовлення №4: 6588.00 грн

Кількість прийнятих замовлень: 2

Загальна виручка: 9180.00 грн

7. Тестування програми

Відповідно до лабораторної роботи потрібно перевірити щонайменше референтну прозорість, відсутність мутації вхідних даних та правильність роботи Callable-політик.

Файл tests/test_core.py:

Запуск тестів:

pytest -q

Очікуваний результат:

..... [100%]
6 passed

8. Використання typing.Callable

Важливою частиною роботи є передача поведінки програми у вигляді функцій. У методичці це прямо визначено як одне із завдань лабораторної роботи.

Наприклад:

```
FilterFn = Callable[[float], bool]
```

```
DiscountFn = Callable[[float], float]
```

```
TaxFn = Callable[[float], float]
```

Тому можна змінювати поведінку програми без зміни make_processor():

```
processor = make_processor(  
    accept=lambda x: x >= 500,  
    apply_discount=lambda x: x * 0.95,  
    apply_tax=lambda x: x * 1.20,  
)
```

Або:

```
processor = make_processor(  
    accept=lambda x: x >= 5000,  
    apply_discount=lambda x: x * 0.80,  
    apply_tax=lambda x: x * 1.10,  
)
```

Це демонструє функціональну параметризацію програми.

9. Аналіз функціонального підходу

У створеній програмі дотримано основних принципів лабораторної роботи.

Чистота функцій. Розрахункові функції не виконують введення-виведення і залежать лише від своїх аргументів.

Відсутність мутацій. Замість:

```
order["total"] = total
```

використовується:

```
return {  
    **order,  
    "total": total,  
}
```

Отже, створюється новий словник.

Референтна прозорість. Виклик:

```
process_orders_pure(orders, ...)
```

для однакових `orders` і параметрів повертає однаковий результат.

Ізоляція побічних ефектів. `print()` використовується тільки в `app.py`, тоді як `core.py` не виконує I/O.

Функції як значення. Політики прийняття замовлення, знижки та податку передаються через `Callable`.

Функція вищого порядку. `make_processor()` отримує функції як аргументи та повертає нову функцію:

```
processor = make_processor(...)  
result = processor(orders)
```

10. Висновок

У лабораторній роботі було розроблено програму функціональної обробки замовлень інтернет-магазину.

Було реалізовано чисте обчислювальне ядро, яке не використовує глобальний змінний стан, не виконує операцій введення-виведення та не модифікує початкові структури даних.

Для параметризації алгоритму застосовано `typing.Callable`. Політики фільтрації замовлень, розрахунку знижки та податку передаються у функцію вищого порядку `make_processor()`, завдяки чому поведінку програми можна змінювати без модифікації її основного алгоритму.

За допомогою `pytest` перевірено правильність обчислень, референтну прозорість, відсутність мутації початкових даних та коректність функціональних політик. Таким чином, практично реалізовано основні принципи функціонального програмування у Python, визначені в лабораторній роботі.